

Paradigmas de Programación

Paradigmas Estructurado / Orientado a Objetos / Funcional



¿Qué es un paradigma de programación?



Paradigmas de Programación

- Existen diversas metodologías para la escritura de código, cada una con un enfoque particular para abordar y resolver problemas de programación.
- Cada paradigma de programación posee sus propias reglas y técnicas distintivas.
- La elección del paradigma influirá en cómo conceptualizamos el problema, diseñamos la solución y escribimos el código.
- El conocimiento de los diferentes paradigmas nos permite seleccionar el enfoque más adecuado para cada situación específica.



Paradigma Estructurado

- Principio de "Divide y Vencerás": El programa se descompone en módulos más pequeños y manejables, denominados funciones.
- Las funciones se estructuran de manera secuencial para lograr un flujo lógico.
- El objetivo es resolver problemas de forma lógica y estructurada.
- Se emplean principalmente estructuras de control secuenciales, selectivas e iterativas.
- En proyectos de gran envergadura, este enfoque puede dificultar la comprensión y el mantenimiento del código.



Paradigma Estructurado

```
public class SumArray {  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3, 4, 5};  
        int sum = calculateSum(numbers);  
        System.out.println("La suma es: " + sum);  
    }  
  
    public static int calculateSum(int[] numbers) {  
        int sum = 0;  
        for (int number : numbers) {  
            sum += number;  
        }  
        return sum;  
    }  
}
```

El paradigma estructurado se caracteriza por la organización secuencial del código, el uso de funciones y la aplicación de estructuras de control como bucles y condicionales. A continuación, te muestro un ejemplo en Java y Python que ilustra este paradigma mediante la resolución de un problema sencillo: calcular la suma de los números de un array.

```
def calculate_sum(numbers):  
    sum = 0  
    for number in numbers:  
        sum += number  
    return sum  
  
if __name__ == "__main__":  
    numbers = [1, 2, 3, 4, 5]  
    sum = calculate_sum(numbers)  
    print(f"La suma es: {sum}")
```

Paradigma Orientado a Objetos

- Se fundamenta en la representación del mundo real mediante la creación de entidades conocidas como objetos. (Abstracción)
- Estos objetos integran datos y comportamientos en una única entidad cohesiva. (Encapsulamiento)
- Los objetos interactúan entre sí a través del intercambio de mensajes.
- Fomenta la reutilización del código y la organización jerárquica de las clases. (Herencia)
- Proporciona flexibilidad y facilita el mantenimiento del código. (Polimorfismo)



Paradigma Orientada a Objetos (POO)

El paradigma orientado a objetos se basa en la creación de clases que encapsulan tanto datos (atributos) como comportamientos (métodos). A continuación, te presento un ejemplo en Java y Python donde se implementa una clase **Círculo** que permite calcular el área y el perímetro de un círculo.

```
class Circulo {
    private double radio;

    // Constructor
    public Circulo(double radio) {
        this.radio = radio;
    }

    // Método para calcular el área
    public double calcularArea() {
        return Math.PI * Math.pow(radio, 2);
    }

    // Método para calcular el perímetro
    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }

    // Método para mostrar la información del círculo
    public void mostrarInformacion() {
        System.out.println("Radio: " + radio);
        System.out.println("Área: " + calcularArea());
        System.out.println("Perímetro: " + calcularPerimetro());
    }

    public static void main(String[] args) {
        Circulo circulo = new Circulo(5.0);
        circulo.mostrarInformacion();
    }
}
```

```
class Circulo:
    def __init__(self, radio):
        self.radio = radio

    def calcular_area(self):
        return 3.14159 * (self.radio ** 2)

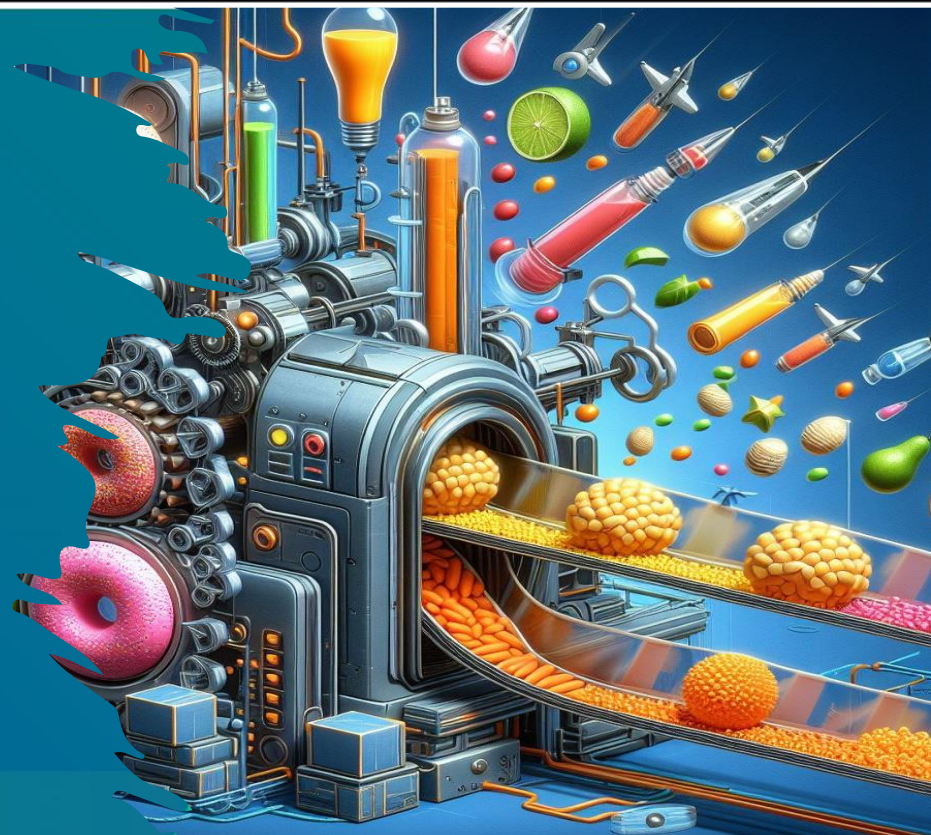
    def calcular_perimetro(self):
        return 2 * 3.14159 * self.radio

    def mostrar_informacion(self):
        print(f"Radio: {self.radio}")
        print(f"Área: {self.calcular_area()}")
        print(f"Perímetro: {self.calcular_perimetro()}")

if __name__ == "__main__":
    circulo = Circulo(5.0)
    circulo.mostrar_informacion()
```

Paradigma Funcional

- Las funciones se consideran entidades de primera clase.
- Pueden ser asignadas a variables.
- Pueden ser pasadas como argumentos a otras funciones.
- Pueden ser retornadas como resultados de otras funciones.
- Este enfoque enfatiza la inmutabilidad de los datos.
- Utiliza la recursión como una técnica clave para realizar operaciones repetitivas.



Paradigma Funcional

El paradigma funcional se centra en la utilización de funciones como ciudadanos de primera clase, la inmutabilidad de los datos y el uso de funciones puras, donde no se generan efectos secundarios. A continuación, te presento un ejemplo en Java y Python que muestra cómo se puede calcular la suma de los cuadrados de una lista de números utilizando un enfoque funcional.

```
import java.util.Arrays;
import java.util.List;

public class SumaCuadrados {
    public static void main(String[] args) {
        List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5);

        // Paradigma funcional: map y reduce
        int sumaDeCuadrados = numeros.stream()
            .map(x -> x * x) // función pura que eleva al cuadrado cada número
            .reduce(0, Integer::sum); // función pura que suma los cuadrados

        System.out.println("La suma de los cuadrados es: " + sumaDeCuadrados);
    }
}
```

```
from functools import reduce

def cuadrado(x):
    return x * x

if __name__ == "__main__":
    numeros = [1, 2, 3, 4, 5]

    # Paradigma funcional: map y reduce
    suma_de_cuadrados = reduce(lambda a, b: a + b, map(cuadrado, numeros))

    print(f"La suma de los cuadrados es: {suma_de_cuadrados}")
```

Paradigma Declarativo

- Se enfoca en especificar qué debe lograrse en lugar de detallar cómo debe implementarse.
- Describe las relaciones y restricciones entre los datos.
- Se abstrae de los pormenores de la implementación específica.
- Prioriza la lógica subyacente del problema en cuestión.
- El código declarativo suele ser más legible y fácil de mantener.



Paradigma Declarativo

El paradigma descriptivo, a menudo asociado con el estilo declarativo, se enfoca en especificar el qué se debe hacer en lugar del cómo hacerlo. A menudo se utiliza en lenguajes de programación que permiten expresar la lógica de manera más abstracta. Un ejemplo clásico en el paradigma descriptivo es el uso de consultas a bases de datos. A continuación, te muestro cómo se puede implementar una consulta descriptiva en Java usando SQL con JDBC y en Python usando una consulta SQL a través de una librería de base de datos.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class ConsultaDescriptiva {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mi_base_de_datos";
        String usuario = "mi_usuario";
        String contrasena = "mi_contrasena";

        try (Connection conn = DriverManager.getConnection(url, usuario, contrasena);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT nombre FROM empleados WHERE salario > 50000")) {

            while (rs.next()) {
                String nombre = rs.getString("nombre");
                System.out.println("Empleado: " + nombre);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

import sqlite3

def consultar_empleados():
    conexion = sqlite3.connect('mi_base_de_datos.db')
    cursor = conexion.cursor()

    # Consulta descriptiva
    cursor.execute("SELECT nombre FROM empleados WHERE salario > 50000")

    for fila in cursor.fetchall():
        print(f"Empleado: {fila[0]}")

    conexion.close()

if __name__ == "__main__":
    consultar_empleados()
```

Resumen

Hemos aprendido que cada paradigma de programación ofrece un enfoque único para la resolución de problemas, con fortalezas y debilidades particulares. Observamos que el paradigma estructurado se centra en la organización secuencial del código, mientras que el paradigma orientado a objetos se basa en la creación de objetos que encapsulan datos y comportamientos, y el paradigma funcional pone énfasis en las funciones como entidades de primera clase y en la inmutabilidad de los datos.

Exploramos cómo la elección del paradigma adecuado puede depender tanto del problema específico que se pretende resolver como de las preferencias personales y las necesidades del proyecto. Finalmente, reconocemos la importancia de comprender estos paradigmas como herramientas esenciales en el repertorio de todo programador, y cómo la combinación adecuada de ellos puede conducir a soluciones de software más robustas, mantenibles y eficientes.

